Oops Assignment - #2

1. The this pointer is passed as a hidden argument to:
   A. All non-static member function calls
   B. All static member function calls
   C. Both static and non-static member function calls
   D. More than one of the above
   E. None of the above

2. Predict the output of the following C++ program:

```
#include<iostream>
using namespace std;
class Test
{ private:
  int x;   int y;
public:
  Test(int x = 0, int y = 0) { this->x = x; this->y = y; }
  static void fun1() { cout << "Inside fun1()"; }
  static void fun2() { cout << "Inside fun2()"; this->fun1(); }
};
int main()
{
  Test obj;
  obj.fun2();
  return 0; }
```

   A. Inside fun2() Inside fun1()
   B. Inside fun2()
   C. Inside fun1() Inside fun2()
   D. More than one of the above
   E. None of the above

3. Predict the output of the following C++ program:

```
#include<iostream>
using namespace std;
class Test
{  private:
  int x;
public:
  Test() { x = 0; }
  void destroy() { delete this; }
  void print() { cout << "x = " << x; } };
int main()
{  Test obj;
  obj.destroy();
  obj.print();
  return 0; }
```

   A. x = 0
   B. Undefined behavior
   C. Compiler error
   D. More than one of the above
   E. None of the above

4. Which of the following statements is true?
   A. All objects of a class share all data members of the class.
   B. Objects of a class do not share non-static members; every object has its own copy.
   C. Objects of a class do not share the code of non-static methods; they have their own copy.
   D. More than one of the above
   E. None of the above

5. Assume an integer and a pointer each take 4 bytes. Also, assume no alignment in objects. Predict the output of the following program:

```
#include<iostream>
using namespace std;
class Test
{
    static int x;
    int *ptr;
    int y;
};
int main()
{
    Test t;
    cout << sizeof(t) << " ";
    cout << sizeof(Test *);
}
```

   A. 12 4
   B. 12 12
   C. 8 4
   D. More than one of the above
   E. None of the above

6. Which of the following cannot be passed to a function in C++?
   A. Constant
   B. Structure
   C. Array
   D. Header file
   E. None of the above

7. When a method in a subclass has the same name and type signatures as a method in the superclass, the method in the subclass _____ the method in the superclass.
   A. Overloads
   B. Overrides
   C. Inherits
   D. More than one of the above
   E. None of the above

8. Which of the following statements about friend functions in C++ is true?
A. Friend functions are a part of the class they are declared in.
B. Friend functions must be declared in the private section of a class.
C. Friend functions can access private and protected members of a class.
D. More than one of the above
E. None of the above

9. What will be the output of the following program?

```
#include<iostream>
using namespace std;
class Base
{
public:
  virtual void show() { cout << "Base"; }
};
class Derived : public Base
{
public:
  void show() { cout << "Derived"; }  };
int main()
{
  Base *b = new Derived();
  b->show();
  return 0;
}
```

A. Base
B. Derived
C. Compiler Error
D. Undefined Behavior
E. None of the above

10. Which of the following is not a feature of object-oriented programming?
A. Encapsulation
B. Abstraction
C. Polymorphism
D. More than one of the above
E. None of the above

11. Which of the following best describes "function overloading"?
A. Defining multiple functions with the same name but different parameter lists.
B. Defining multiple functions with the same name but different return types
C. Using a single function for multiple purposes
D. More than one of the above
E. None of the above

12. Which of the following operators cannot be overloaded in C++?
A. +
B. =
C. ::
D. []
E. None of the above

13. What is the primary difference between a shallow copy and a deep copy?
A. A shallow copy duplicates the entire object, including dynamically allocated memory.
B. A deep copy duplicates only the object's static members.
C. A shallow copy duplicates only the object's pointer, while a deep copy duplicates the memory pointed to by the pointer.
D. More than one of the above
E. None of the above

14. In C++, a pure virtual function is:
A. A function with no definition in the base class
B. A function that must be overridden in the derived class
C. Declared using the = 0 syntax
D. More than one of the above
E. None of the above

15. Which of the following is true about destructors in C++?
A. A class can have more than one destructor.
B. Destructors are called explicitly to free memory.
C. Destructors can be overloaded.
D. Destructors cannot be virtual.
E. None of the above

16. Which of the following is true?
A. Static methods cannot be overloaded.
B. Static data members can only be accessed by static methods.
C. Non-static data members can be accessed by static methods.
D. More than one of the above
E. None of the above

17. Predict the output of the following C++ program:

```
#include <iostream>
using namespace std;
class Test
{    static int x;
public:
   Test() { x++; }
   static int getX() { return x; } };
int Test::x = 0;
int main()
{    cout << Test::getX() << " ";
   Test t[5];
   cout << Test::getX(); }
```

A. 0 0
B. 5 5
C. 0 5
D. More than one of the above
E. None of the above

18. What will be the output of the following C++ program?

```
#include<iostream>
using namespace std;
int fun(int x = 0, int y = 0, int z)
{     return (x + y + z);  }
int main()
{   cout << fun(10);
  return 0;  }
```

A. 10
B. 0
C. 20
D. More than one of the above
E. None of the above

19. What is the output of the following C++ program?

```
#include <iostream>
using namespace std;
void square(int *x)
{       *x = (*x)++ * (*x);    }
void square(int *x, int *y)
{       *x = (*x) * --(*y);      }
int main()
{
   int number = 30;
   square(&number, &number);
   cout << number;
   return 0;    }
```

A. 910
B. 920
C. 870
D. More than one of the above
E. None of the above

20. Assume that an integer and a pointer each take 4 bytes. Also, assume that there is no alignment in objects. Predict the output of the following program.

```
#include<iostream>
using namespace std;
class Test
{   static int x;
   int *ptr;
   int y;
};
int main()
{   Test t;
   cout << sizeof(t) << " ";
   cout << sizeof(Test *);  }
```

A. 12 4
B. 12 12
C. 8 4
D. More than one of the above
E. None of the above

21. Which of the following is not a correct statement regarding abstract classes in C++?
A. Every class containing an abstract method must be declared abstract.
B. Abstract class can directly be instantiated with the new operator.
C. Abstract class can be instantiated.
D. More than one of the above.
E. None of the above.

22. It is possible to define a class within another class, which is called a nested class. How many types of nested classes exist in C++?
A. 2
B. 3
C. 4
D. More than one of the above
E. None of the above

23. When one object reference variable is assigned to another object reference variable, what happens?
A. A copy of the object is created.
B. A copy of the reference is created.
C. A copy of the reference is not created.
D. More than one of the above.
E. None of the above.

24. What will be the output of the following C++ program?

```
#include <iostream>
using namespace std;
class Player
{
private:
   int id;
   static int next_id;
public:
   int getID() { return id; }
   Player() { id = next_id++; }
};
int Player::next_id = 1;
int main()
{
   Player p1, p2, p3;
   cout << p1.getID() << " ";
   cout << p2.getID() << " ";
   cout << p3.getID();
   return 0;
}
```

A. Compiler Error
B. 1 2 3
C. 1 1 1
D. More than one of the above
E. None of the above

25. What will be the output of the following C++ program?

```
#include <iostream>
class Test {
public:
   void fun();
};
static void Test::fun() {
   std::cout << "fun() is static\n";
}
int main() {
   Test::fun();
   return 0;
}
```

A. fun() is static
B. Empty Screen
C. Compiler Error
D. More than one of the above
E. None of the above

26. What will be the output of the following C++ program?

```
#include<iostream>
using namespace std;
class Test
 {
private:
   static int count;
public:
   Test& fun();
};
int Test::count = 0;
Test& Test::fun() {
   Test::count++;
   cout << Test::count << " ";
   return *this;
}
int main()
{
   Test t;
   t.fun().fun().fun().fun();
   return 0;
}
```

A. Compiler Error
B. 1 2 3 4
C. 1 1 1 1
D. More than one of the above
E. None of the above

27. Predict the output of the following C++ program:

```
#include <iostream>
using namespace std;
class Test {
   static int x;
public:
   Test() { x++; }
   static int getX() { return x; }
};
int Test::x = 0;
int main() {
   cout << Test::getX() << " ";
   Test t[5];
   cout << Test::getX();
}
```

A. 0 0
B. 5 5
C. 0 5
D. More than one of the above
E. None of the above

28. When the inheritance is private, the private methods in base class are _____ in the derived class.
A) Inaccessible
B) Accessible
C) Protected
D) More than one of the above
E) None of the above

29. What will be the output of the following code?
```
#include<iostream>
using namespace std;
class P {
public:
  void print() { cout << "Inside P"; } };
class Q : public P {
public:
  void print() { cout << "Inside Q"; } };
class R : public Q { };
int main() {
 R r;
 r.print();
 return 0;  }
```
A) Inside P
B) Inside Q
C) Compiler Error: Ambiguous call to print()
D) More than one of the above
E) None of the above

30. What will be the output of the following destructor program?
```
#include<iostream>
using namespace std;
class Base1 {
public:
  ~Base1() { cout << "Base1's destructor" << endl; } };
class Base2 {
public:
  ~Base2() { cout << "Base2's destructor" << endl; } };
class Derived : public Base1, public Base2 {
public:
  ~Derived() { cout << "Derived's destructor" << endl; } };
int main() {
  Derived d;
  return 0;  }
```
A) Base1's destructor, Base2's destructor, Derived's destructor
B) Derived's destructor, Base2's destructor, Base1's destructor
C) Derived's destructor
D) More than one of the above
E) None of the above

31. Identify the compiler error in the following code:
```
#include<iostream>
using namespace std;
class Base {};
class Derived : public Base {};
int main() {
  Base *bp = new Derived;
  Derived *dp = new Base;  }
```
A) No Compiler Error
B) Compiler Error in line "Base *bp = new Derived;"
C) Compiler Error in line "Derived *dp = new Base;"
D) More than one of the above
E) None of the above

32. What will be the output of the following code?
```
#include <iostream>
using namespace std;
int main() {
   const char* p = "12345";
   const char **q = &p;
   *q = "abcde";
   const char *s = ++p;
   p = "XYZWVU";
   cout << *++s;
   return 0; }
```
A) Compiler Error
B) c
C) b
D) More than one of the above
E) None of the above

33. In C++, the const qualifier can be applied to which of the following?
1. Member functions of a class
2. Function arguments
3. Class data members declared as static
4. Reference variables

A) Only 1 and 2
B) Only 3 and 4
C) only 2 and 4
D) More than one of the above
E) None of the above

34. What will be the output of the following code?

```
#include <stdio.h>
int main() {
 const int x;
 x = 10;
 printf("%d", x);
return 0; }
```

A) Compiler Error
B) 10
C) 0
D) More than one of the above
E) None of the above

35. What will be the output of the following code?

```
#include <iostream>
int const s = 9;
int main() {
   std::cout << s;
   return 0;  }
```

A) 9
B) Compiler Error
C) 0
D) More than one of the above
E) None of the above

36. Which of the following is true about virtual functions in C++?

A) Virtual functions can be overridden in derived classes with the same signature.
B) Virtual functions enable run-time polymorphism in an inheritance hierarchy.
C) If a function is virtual in the base class, the most-derived class's implementation is called according to the actual type of the object referred to.
D) More than one of the above
E) None of the above

37. What will be the output of the following code?

```
#include<iostream>
using namespace std;
class Base {
public:
   virtual void show() { cout << "In Base\n"; }  };
class Derived : public Base {
public:
   void show() { cout << "In Derived\n"; }  };
int main() {
   Base *bp = new Derived;
   bp->show();
   Base &br = *bp;
   br.show();
   return 0;  }
```

A) In Base In Base
B) In Base In Derived
C) In Derived In Derived
D) More than one of the above
E) None of the above

38. Which of the following is true about pure virtual functions in C++?

1. Their implementation is not provided in the class where they are declared.
2. If a class has a pure virtual function, then the class becomes an abstract class, and an instance of this class cannot be created.

A) None
B) Only 1
C) Only 2
D) More than one of the above (B and C)
E) None of the above

39. What will be the output of the following code?

```
#include<iostream>
using namespace std;
class Base {
public:
   virtual void show() = 0;  // Pure virtual function
};
int main(void) {
   Base b;  // Attempting to create an instance of an
abstract class
   Base *bp;
   return 0;  }
```

A) Compiler errors in lines "Base b;" and "Base *bp;"
B) Compiler error in line "Base b;"
C) Compiler error in line "Base *bp;"
D) More than one of the above
E) None of the above

40. What will be the output of the following code?

```
#include<iostream>
using namespace std;
class Base {
public:
   virtual void show() = 0;  // Pure virtual function
};
class Derived : public Base { };
int main(void) {
   Derived q;  // Attempting to instantiate Derived
without overriding the pure virtual function
   return 0;
}
```

A) Compiler Error: There cannot be an empty derived class
B) Compiler Error: Derived is abstract
C) No compiler Error
D) More than one of the above
E) None of the above

41. What will be the output of the following code?

```
#include <iostream>
using namespace std;
class A {
public:
   virtual void fun() { cout << "A::fun() "; }
};
class B: public A {
public:
   void fun() { cout << "B::fun() "; }
};
class C: public B {
public:
   void fun() { cout << "C::fun() "; }
};
int main() {
   B *bp = new C;
   bp->fun();
   return 0;
}
```

A) A::fun()
B) B::fun()
C) C::fun()
D) More than one of the above
E) None of the above

42. What will be the output of the following code?

```
#include<iostream>
using namespace std;
class Base {
public:
   virtual void show() { cout << "In Base\n"; }
};
class Derived: public Base {
public:
   void show() { cout << "In Derived\n"; }
};
int main(void) {
   Base *bp = new Derived;
   bp->Base::show();  // Note the use of scope resolution
here
   return 0;
}
```

A) In Base
B) In Derived
C) Compiler Error
D) More than one of the above
E) None of the above

43. How to create a dynamic array of pointers (to integers) of size 10 using new in C++?

A) int **arr = new int *[10];
B) int *arr = new int [10];
C) int *arr = new int *[10];
D) More than one of the above
E) None of the above

44. What happens when delete is used for a NULL pointer?

```
int *ptr = NULL;
delete ptr;
```

A) Compiler Error
B) Run-time Crash
C) No Effect
D) More than one of the above
E) None of the above

45. What is the advantage of exception handling in C++?

1. Remove error-handling code from the software's main line of code.
2. A method writer can choose to handle certain exceptions and delegate others to the caller.
3. An exception that occurs in a function can be handled anywhere in the function call stack.

A) Only 1
B) Only 2
C) Only 3
D) More than one of the above
E) None of the above

46. What should be put in a try block in C++?
1. Statements that might cause exceptions
2. Statements that should be skipped in case of an exception

A) Only 1
B) Only 2
C) None
D) More than one of the above
E) None of the above

47. Which of the following is true about virtual functions in C++?
A) Virtual functions allow dynamic binding at runtime.
B) Virtual functions must be defined in every derived class.
C) Virtual functions cannot be accessed using a base class pointer.
D) More than one of the above
E) None of the above

48. What will happen if a pure virtual function is not overridden in a derived class in C++?
A) Compiler Error when creating an object of the derived class
B) Program will compile but crash at runtime
C) Base class constructor will fail
D) More than one of the above
E) None of the above

49. What will be the output of the following C++ code?
```
#include <iostream>
using namespace std;
class Base {
public:
   Base() { cout << "Base Constructor "; }
   virtual ~Base() { cout << "Base Destructor "; }  };
class Derived : public Base {
public:
   Derived() { cout << "Derived Constructor "; }
   ~Derived() { cout << "Derived Destructor "; }  };
int main() {
   Base *ptr = new Derived();
   delete ptr;
   return 0;  }
```
A) Base Constructor Derived Constructor Base Destructor
B) Base Constructor Derived Constructor Derived Destructor
C) Base Constructor Derived Constructor Derived Destructor Base Destructor
D) More than one of the above
E) None of the above

50. What will be the output of the following code?
```
#include<iostream>
using namespace std;
int main() {
   int *ptr = new int(5);
   cout << *ptr;
   return 0;
}
```
A) 0
B) 5
C) Garbage Value
D) More than one of the above
E) None of the above

51. What will happen when you run the following code?
```
#include <iostream>
using namespace std;
int fun(int a, int b = 1, int c = 2) {
   return (a + b + c); }
int main() {
   cout << fun(12, , 2);
   return 0; }
```
A) 15
B) 16
C) Compiler Error
D) More than one of the above
E) None of the above

52. What will be the output of the following code?
```
#include <iostream>
#include <string.h>
using namespace std;
int main() {
   cout << sizeof("abcdefghijklm") << endl;
   cout << strlen("abcdefghijklm ");
   return 0;
}
```
A) 13 13
B) 14 13
C) 13 14
D) More than one of the above
E) None of the above

53. What will be the output of the following code?

```
#include<iostream>
using namespace std;
int main() {
   int x = 1, y = 1, z = 1;
   cout << (++x || ++y && ++z) << endl;
   cout << x << " " << y << " " << z;
   return 0;
}
```

A) 1 2 1 1
B) 1 2 2 2
C) 0 1 1 1
D) More than one of the above
E) None of the above

54. What will be the output of the following program?

```
#include<iostream>
using namespace std;
int x; // Global x
int main() {
   int x = 10; // Local x
   cout << "Value of global x is " << ::x << endl;
   cout << "Value of local x is " << x;
   return 0;
}
```

A) Value of global x is 0
Value of local x is 10
B) Value of global x is 10
Value of local x is 0
C) Value of global x is Garbage
Value of local x is 10
D) More than one of the above
E) None of the above

55. What will be the output of the following code?

```
#include <iostream>
using namespace std;
int main() {
   for (; ; )
      cout << "blank";
   return 0;
}
```

A) blank (printed once)
B) blank (printed twice)
C) Infinite Loop
D) More than one of the above
E) None of the above

56. What will be the output of the following code?

```
#include <iostream>
using namespace std;
 int main()
{     int i;
  for (i = 0; i < 3; i++);
  cout << "hello!" << i;
  return 0;  }
```

A. hello!2
B. hello!4
C. hello!3
D. More than one of the above
E. None of the above

57. What will be the output of the following code?

```
#include <iostream>
using namespace std;
 int main()
{     int i;
    i = 1 + (1, 4, 5, 6, 3);
    cout << i;
    return 0;  }
```

A. 5
B. 4
C. 3
D. More than one of the above
E. None of the above

58. What will be the output of the following code?

```
#include <iostream>
using namespace std;
 int main()
{     int a = 0, b;
    b = (a = 50) + 10;
    cout << a << "$" << b;
    return 0; }
```

A. 0$60
B. 50$50
C. 50$60
D. More than one of the above
E. None of the above

59. What will be the output of the following code?

```
#include<iostream>
using namespace std;
 int main()
{    char a = 30, b = 40, c = 10;
   char d = (a * b) / c;
   cout << int(d);
   return 0;  }
```

A. 120
B. 100
C. 90
D. More than one of the above
E. None of the above

60. What will be the output of the following code?

```
#include<iostream>
using namespace std;
 int main(int x)
{    static int i = 5;
    if (--i)
   {       cout << i;
     main(10);     }
  return 0;  }
```

A. 5
B. 4321
C. Infinite Loop
D. More than one of the above
E. None of the above

61. What will be the output of the following code?

```
#include<iostream>
using namespace std;
 int main(int x)
{    int i = 5;
    if (--i)     {
      cout << i;
      main(10);     }
   return 0; }
```

A. 4321
B. 54321
C. Infinite Loop
D. More than one of the above
E. None of the above

62. What will be the output of the following code?

```
#include <iostream>
using namespace std;
int main()
{   int x = 0;
   for (int i = 0; i < 10; i++)
   {       x = x++ + ++x;
     if (x > 100)
       break;    }
   cout << x;
   return 1;   }
```

A. 100
B. Compiler Error
C. Compiler Dependent
D. More than one of the above
E. None of the above

63. How many times will the cout statement execute in the following code?

```
#include <iostream>
using namespace std;
int main()
{    int n = 10;
   for (int i = 0; i < n; i++)
   {       n++;
     continue;
     cout << n;    }
   return 1; }
```

A. 10
B. 0
C. Infinite Loop
D. More than one of the above
E. None of the above

64. What will be the output of the following code?

```
#include <iostream>
using namespace std;
int main()
{    int n = 10, i;
   for (i = 0; i < n; i++)
   {       n++;
     cout << n << endl;
     goto x;    }
x:
   do
   {       cout << "label x" << endl;
     break;    } while (0);
   return 1; }
```

A. 11
B. 11 followed by "label x"
C. "label x" only
D. More than one of the above
E. None of the above

65. What will be the output of the following code?

```
#include <iostream>
using namespace std;
int main()
{
   int choice = 1;
   switch (choice)
   {
   case 1:
      cout << "\nInside Switch case 1";
   case 2:
      cout << "\nInside Switch case 2";
      break;
   case 3:
      cout << "\nInside Switch case 3";
      break;
   default:
      cout << "bye bye";
   }
   return 0;
}
```

A. Inside Switch case 1
B. Inside Switch case 1
   Inside Switch case 2
C. bye bye
D. More than one of the above
E. None of the above

66. Which of the following statements is correct?
A) Base class pointer cannot point to derived class.
B) Derived class pointer cannot point to base class.
C) Pointer to derived class cannot be created.
D) More than one of the above.
E) None of the above.

67. Why is a reference not the same as a pointer?
A) A reference can never be null.
B) A reference, once established, cannot be changed.
C) A reference doesn't need an explicit dereferencing mechanism.
D) More than one of the above.
E) None of the above.

68. Which of the following statements is correct?
A) A constructor is called at the time of declaration of an object.
B) A constructor is called at the time of use of an object.
C) A constructor is called at the time of declaration of a class.
D) More than one of the above.
E) None of the above.

69. Which of the following cannot be used with the keyword virtual?
A) class
B) member functions
C) constructor
D) More than one of the above
E) None of the above

70. Which of the following functions are performed by a constructor?
A) Construct a new class
B) Construct a new object
C) Construct a new function
D) More than one of the above
E) None of the above

71. Which of the following problems causes an exception?
A) Missing semicolon in a statement in main().
B) A problem in the calling function.
C) A syntax error.
D) More than one of the above.
E) None of the above.

72. The compiler checks the type of reference in the object and not the type of object. This concept refers to:
A) Inheritance
B) Polymorphism
C) Abstraction
D) More than one of the above
E) None of the above

73. Which of the following is the correct way of declaring a function as constant?
A) const int ShowData(void) { /* statements */ }
B) int const ShowData(void) { /* statements */ }
C) int ShowData(void) const { /* statements */ }
D) More than one of the above
E) None of the above

74. Which of the following concepts is used to implement late binding?
A) Virtual function
B) Operator function
C) Const function
D) More than one of the above
E) None of the above

75. Which of the following statements is correct?
A) C++ allows static type checking.
B) C++ allows dynamic type checking.
C) C++ allows static member functions to be of type const.
D) More than one of the above.
E) None of the above.

76. Which of the following factors support the statement that reusability is a desirable feature of a language?
A) It decreases the testing time.
B) It lowers the maintenance cost.
C) It reduces the compilation time.
D) More than one of the above.
E) None of the above.

77. Which of the following ways are legal to access a class data member using the this pointer?
A) this->x
B) this.x
C) *this.x
D) More than one of the above
E) None of the above

78. Which of the following is a mechanism of static polymorphism?
A) Operator overloading
B) Function overloading
C) Templates
D) More than one of the above
E) None of the above

79. Which of the following is correct about the statements given below?
1.All operators can be overloaded in C++.
2.We can change the basic meaning of an operator in C++.
A) Only I is true
B) Both I and II are false
C) Only II is true
D) More than one of the above
E) None of the above

80. What happens if the base and derived class contain definitions of a function with the same prototype?
A) Compiler reports an error on compilation.
B) Only the base class function will get called irrespective of the object.
C) Only the derived class function will get called irrespective of the object.
D) Base class object will call the base class function, and derived class object will call the derived class function.
E) None of the above

81. Which of the following are available only in the class hierarchy chain?
A) Public data members
B) Private data members
C) Protected data members
D) More than one of the above
E) None of the above

82. In which of the following cases is a virtual call resolved at the time of compilation?
A) From inside the destructor
B) From inside the constructor
C) From inside the main() function
D) More than one of the above
E) None of the above

83. Which of the following statements regarding inline functions is correct?
A) It speeds up execution.
B) It slows down execution.
C) It increases the code size.
D) More than one of the above
E) None of the above

84. Which one of the following is the correct way to declare a pure virtual function?
A) virtual void Display(void) {0};
B) virtual void Display = 0;
C) virtual void Display(void) = 0;
D) More than one of the above
E) None of the above

85. Which of the following header files includes the definition of cin and cout?
A) istream.h
B) ostream.h
C) iomanip.h
D) More than one of the above
E) None of the above

86. Which of the following keywords is used to overload an operator?
A) overload
B) operator
C) friend
D) More than one of the above
E) None of the above

87. What will happen if a class is not having any name?
A) It cannot have a destructor.
B) It cannot have a constructor.
C) It is not allowed.
D) More than one of the above
E) None of the above

88. Which inheritance type is used in the class given below?
class A : public X, public Y {};
A) Multilevel inheritance
B) Multiple inheritance
C) Hybrid inheritance
D) More than one of the above
E) None of the above

89. Which one of the following is correct about the statements given below?
1. All function calls are resolved at compile-time in Procedure Oriented Programming.
2. All function calls are resolved at compile-time in OOPS.
A) Only II is correct
B) Both I and II are correct
C) Only I is correct
D) More than one of the above
E) None of the above

90. Which of the following is an invalid visibility label while inheriting a class?
A) public
B) private
C) protected
D) More than one of the above
E) None of the above

91. Which of the following statements is correct in C++?
A) Classes cannot have data as protected members.
B) Structures can have functions as members.
C) Class members are public by default.
D) More than one of the above
E) None of the above

92. Which of the following is used to make an abstract class?
A) Declaring it abstract using static keyword
B) Declaring it abstract using virtual keyword
C) Making at least one member function a virtual function
D) Making at least one member function a pure virtual function
E) None of the above

93. Which of the following access specifiers is used as default in a class definition?
A) protected
B) public
C) private
D) More than one of the above
E) None of the above

94. What is correct about the static data member of a class?
A) A static member function can access only static data members of a class.
B) A static data member is shared among all the objects of the class.
C) A static data member can be accessed directly from main().
D) More than one of the above
E) None of the above

95. When does the "diamond problem" occur in programming?
A) When multiple interfaces are implemented
B) When a class inherits from two classes with the same method
C) When multiple classes inherit from a single class
D) More than one of the above
E) None of the above

96. A programmer defines a class but fails to create an instance of that class. They try to call a method of the class directly. What type of error will occur?
A) Compilation error
B) Runtime error
C) Logical error
D) More than one of the above
E) None of the above

97. Which of the following best describes the concept of "state" in an object?
A) The behavior the object exhibits
B) The methods the object can execute
C) The data stored within the object
D) More than one of the above
E) None of the above

98. What is an object in programming?
A) A piece of data
B) A runtime entity
C) A template for creating objects
D) More than one of the above
E) None of the above

99. What is the main difference between a class and an object?

A) A class is a blueprint, while an object is an instance of a class

B) A class is executable code, while an object is not

C) A class cannot exist without objects, while an object can exist without classes

D) More than one of the above

E) None of the above

100. What is the result of compiling and running the following code in Java?

```
class Counter {
   private: int count = 0;
   void increment() {
      count++;
   }
   int getCount() {
      return count;
   }
}
```

A) Compilation error

B) Runtime error

C) 0

D) More than one of the above

E) None of the above

Answer With Explanation

1. Answer: A. All non-static member function calls

Explanation: The this pointer is a constant pointer passed as a hidden argument to all non-static member function calls. It refers to the memory address of the calling object. Static member functions, however, do not receive the this pointer because they can be called without an instance of the class. Static functions belong to the class rather than any object, so there is no object context to provide the this pointer.

2. Answer: E. None of the above (Compiler error)

Explanation: The program results in a compiler error. Static member functions cannot use the this pointer because they are not tied to an instance of the class. The call to this->fun1() inside fun2() is invalid. Instead, fun1() should have been called directly as fun1() or Test::fun1().

3. Answer: B. Undefined behavior

Explanation: The behavior of the program is undefined. The delete this statement is only valid if the object was created using new. Since obj was created on the stack, calling delete this leads to undefined behavior. Accessing the object after deletion can result in a crash or unpredictable output.

4. Answer: B. Objects of a class do not share non-static members; every object has its own copy.

Explanation: Non-static data members are specific to each instance of a class, meaning each object has its own copy. Static members, however, are shared across all instances of the class. Objects do share the same code for methods (whether static or non-static), as the code resides in a single memory location.

5. Answer: C. 8 4

Explanation: Static members are not part of the object's size as they are shared among all instances of the class. Therefore, the size of t is determined by the non-static members: ptr (4 bytes) and y (4 bytes), totaling 8 bytes. The size of a pointer (Test *) is 4 bytes on most systems.

6. Answer: D. Header file

Explanation: A header file cannot be passed to a function. Constants, structures, and arrays can all be passed to functions as arguments. Header files are meant to be included in the source code and are not runtime entities.

7. Answer: B. Overrides

Explanation: When a subclass defines a method with the same name and signature as a method in its superclass, it overrides the superclass method. This is a key feature of polymorphism, enabling dynamic method binding at runtime. Overloading, by contrast, involves methods with the same name but different parameter lists.

8. Answer: C. Friend functions can access private and protected members of a class.

Explanation: Friend functions are not part of the class they are declared in but are given special access to the private and protected members of the class. They can be declared anywhere in the class (public, private, or protected sections). They are defined outside the class using the friend keyword in the class declaration.

9. Answer: B. Derived

Explanation: The program demonstrates runtime polymorphism using virtual functions. Since show() in the Base class is declared as virtual, the show() method of the Derived class is called, even though the pointer b is of type Base. This is due to dynamic binding.

10. Answer: E. None of the above

Explanation: OOP focuses on encapsulation, abstraction, polymorphism, and inheritance, which allow modeling real-world entities and relationships.

11. Answer: A. Defining multiple functions with the same name but different parameter lists

Explanation: Function overloading allows defining multiple functions with the same name but different parameter lists (type, number, or order of parameters). Return types are not considered in function overloading.

12. Answer: C. ::
Explanation: The :: (scope resolution) operator cannot be overloaded in C++. Other operators, such as +, =, and [], can be overloaded. Certain operators like . and .* also cannot be overloaded.

13. Answer: C. A shallow copy duplicates only the object's pointer, while a deep copy duplicates the memory pointed to by the pointer.
Explanation: A shallow copy duplicates the object's pointer but not the memory it points to, leading to shared ownership of the same memory. A deep copy, on the other hand, duplicates the dynamically allocated memory as well, ensuring that each object has its own copy of the data.

14. Answer: D (B and C)
Explanation: A pure virtual function is declared in the base class using the = 0 syntax and has no definition in the base class. It must be overridden in any concrete derived class. Pure virtual functions are used to create abstract classes.

15. Answer: E. None of the above
Explanation: A class can have only one destructor, and it cannot be overloaded. Destructors are called implicitly when an object goes out of scope or is deleted. They can be declared as virtual to ensure proper cleanup in inheritance hierarchies.

16. Answer: E. None of the above
Explanation: Static methods in C++ can be overloaded like any other method. However, they are not associated with any object and can only access static members (data and methods) of the class.
static methods can be overloaded. static data members can be accessed by both static and non-static methods. static methods cannot access non-static data members directly, as they do not operate on an instance of the class. Static methods are useful for operations that do not depend on instance-specific data and are shared across all instances of the class.

17. Answer: C. 0 5
Explanation: Static members are shared among all objects of a class and are initialized only once. The static variable x is initialized to 0 using int Test::x = 0;.
The first call to Test::getX() happens before any object of Test is created. At this point, x is still 0, so the first output is 0.
When the array Test t[5] is created, the constructor is called 5 times (once for each element in the array). Each time the constructor is called, the static variable x is incremented by After the array creation, x has a value of 5. The second call to Test::getX() returns 5.
Thus, the output of the program is 0 5.

18. Answer: E (Compiler Error)
Explanation: In C++, all default arguments must be the rightmost arguments in a function parameter list. Here, fun(int x = 0, int y = 0, int z) declares default values for x and y but not for z. This results in a compiler error since z is a required argument and is missing when calling fun(10).
The correct way to define this function would be:
int fun(int x, int y = 0, int z = 0)
This modification ensures all default arguments are on the right side.
Hence, the program fails to compile.

19. Answer: C. 870
Explanation: The function square(int *x, int *y) is called with both arguments pointing to number.
Initially, number = 30.
--(*y) decreases the value from 30 to 29 before multiplication.
The multiplication operation is:

*x = (*x) * (--(*y)) = 30 * 29 = 870
Hence, the final output is 870.

20. Answer: C. 8 4
Explanation: sizeof(Test *) returns 4 since a pointer takes 4 bytes.
sizeof(t) includes:
A pointer variable ptr (4 bytes)
An integer variable y (4 bytes)
The static variable x is not included in the object size as static members are class variables, not instance variables.
Total size: 4 (ptr) + 4 (y) = 8 bytes.

21. Answer: D (More than one of the above)B and C
Explanation: If a class contains a pure virtual function, it must be declared as an abstract class. An abstract class cannot be instantiated directly using new because it contains at least one pure virtual function. Abstract classes cannot be instantiated at all. They can only be used as base classes.
Since both B and C are incorrect, the correct answer is D (More than one of the above).

22. Answer: A. 2
Explanation: In C++, there are two types of nested classes:
Dependent Nested Classes: The inner class uses members of the outer class.
Independent Nested Classes: The inner and outer class do not interact with each other.
Since the number of types is fixed at 2, the correct answer is A.

23. Answer: B (A copy of the reference is created)
Explanation: In C++, assigning one object reference to another does not create a new object.
Instead, it copies the reference to the same object.
Changes made through one reference affect the other since they both point to the same memory.
This is particularly important in shallow copies versus deep copies.
Hence, the correct answer is B (A copy of the reference is created).

24. Answer: B. 1 2 3
Explanation: The static variable next_id is shared across all instances of the Player class.
Each time a new Player object is created, the constructor increments next_id and assigns the value to id.
Execution steps:
p1: id = 1, then next_id becomes 2.
p2: id = 2, then next_id becomes 3.
p3: id = 3, then next_id becomes 4.
Hence, the output is 1 2 3.

25. Answer: C. Compiler Error
Explanation: The error arises because non-static member functions cannot be called without an object. Additionally, you cannot use the static keyword in the function definition outside the class unless the function is also declared static inside the class declaration.
Correct version:

```
class Test {
public:
   static void fun();
};
void Test::fun() {
   std::cout << "fun() is static\n";
```

}
This fixes the issue because now fun() is explicitly declared as static.
Hence, the original program will not compile, leading to a compiler error.

26. Answer: B. 1 2 3 4
Explanation: fun() returns a reference to the same object, allowing chained calls like t.fun().fun().fun().
The static variable count ensures that the same counter is incremented across all calls.
Execution:
First call: count = 1
Second call: count = 2
Third call: count = 3
Fourth call: count = 4
Thus, the output is 1 2 3 4.

27. Answer: C. 0 5
Explanation: The static variable x is initialized to 0.
First cout << Test::getX() outputs 0 since no object has been created yet.
Creating an array t[5] invokes the constructor 5 times, incrementing x each time.
After the array is created, x = 5.
Final output: 0 5.

28. Answer: A) Inaccessible
Explanation: In C++, when inheritance is private, all public and protected members of the base class become private members of the derived class. However, private members of the base class are not accessible directly in the derived class, regardless of the inheritance type (public, protected, or private). They remain encapsulated within the base class, and the derived class cannot access them directly. To access such members, public or protected getter/setter functions should be used if available.

29. Answer: B) Inside Q
Explanation:In this code, R inherits from Q, which overrides the print() method of P. Since R does not override print(), the method from its immediate base class Q is called when r.print() is executed. This behavior is due to method hiding in C++ (not polymorphism since no virtual keyword is used). Therefore, the output will be "Inside Q".

30. Answer: B) Derived's destructor, Base2's destructor, Base1's destructor
Explanation: When an object of a derived class is destroyed, destructors are called in the reverse order of the construction:

1. Derived's destructor is called first.
2. Then Base2's destructor (because Base2 is listed after Base1 in the inheritance list).
3. Finally, Base1's destructor.

This reverse order ensures proper resource deallocation. Hence, the output will be:
Derived's destructor
Base2's destructor
Base1's destructor

31. Answer: C) Compiler Error in line "Derived *dp = new Base;"
Explanation: In C++, upcasting (Base* to Derived object) is valid and allowed without explicit casting, as shown in Base *bp = new Derived;. However, downcasting (Derived *dp = new Base;) is not allowed without an explicit cast because a base class object does not contain the additional data members or behaviors of the derived class. This will result in a compiler error. To downcast safely, dynamic_cast should be used in conjunction with polymorphism (virtual functions).

32. Answer: B) c
Explanation: const char* p = "12345"; points p to the string literal "12345".
const char **q = &p; makes q point to p.

*q = "abcde"; changes the address stored in p to point to "abcde".
const char *s = ++p; increments p to point to "bcde", and assigns this to s.
p = "XYZWVU"; changes p to point to "XYZWVU", but this does not affect s, which still points to "bcde".
cout << *++s; increments s to point to "cde" and prints the first character, which is 'c'.
Hence, the output is 'c'.

33. Answer: D) More than one of the above 1,2,3 and 4
Explanation: Member Functions: Declaring a member function as const ensures it doesn't modify any member variables (unless they are marked mutable).
Function Arguments: const can prevent modification of arguments passed by reference or pointer.
Static Data Members: Although rare, const can be used with static members to define constant values across instances.
Reference Variables: A reference can be const, meaning the data it refers to cannot be modified through the reference.
Thus, all the options are correct.

34. Answer: A) Compiler Error
Explanation: In C/C++, a const variable must be initialized at the time of declaration. Since const int x; is declared without initialization, assigning x = 10; later causes a compiler error. This is because const implies that the value cannot be changed after initialization—and here, it's uninitialized initially, which is invalid.

35. Answer: A) 9
Explanation: The const qualifier can appear before or after the data type, e.g., const int s = 9; or int const s = 9;. Both are valid. Since s is initialized with 9, and its value is not modified afterward, the code compiles and runs successfully, printing 9.

36. Answer: D) More than one of the above (all are true)
Explanation: Virtual functions allow overriding in derived classes.
Virtual functions support run-time polymorphism, enabling dynamic method dispatch based on the object's actual type.
When a virtual function is called via a base class pointer or reference, the function corresponding to the actual object type is executed.
Since all A, B, and C are true, the correct option is D (More than one of the above).

37. Answer: C) In Derived In Derived
Explanation: The show() function in Base is declared as virtual, which enables dynamic dispatch (run-time polymorphism).
bp->show(); calls the Derived class's version because bp points to a Derived object.
br.show(); also calls the Derived class's version because br is a reference to a Derived object (though declared as Base &).
Since both calls are resolved at run-time to the Derived implementation, the output is:
In Derived
In Derived

38. Answer: D) More than one of the above
Explanation: A pure virtual function is declared using = 0 syntax in the base class, and it has no implementation in the class where it's declared. For example:

```
class Base {
   virtual void show() = 0; // Pure virtual function
};
```

39. Answer: B) Compiler error in line "Base b;"
Explanation:Since Base contains a pure virtual function, it is an abstract class.
You cannot instantiate an abstract class directly (Base b; causes a compiler error).
However, pointers to abstract classes (Base *bp;) are allowed because pointers themselves do not create an instance of the class.
Thus, only line Base b; causes a compiler error.

40. Answer: B) Compiler Error: Derived is abstract
Explanation: Base is an abstract class because it has a pure virtual function show().
The Derived class does not override the pure virtual function from Base.
Therefore, Derived also becomes an abstract class, and cannot be instantiated.
Trying to create an object of an abstract class (Derived q;) causes a compiler error.

41. Answer: C) C::fun()
Explanation: The pointer bp is of type B* but points to an object of class C.
The function fun() in class A is declared as virtual, which means the derived class versions will also be treated as virtual.
Even though fun() in B and C are not explicitly marked virtual, they are still virtual due to inheritance from A.
At runtime, the actual type of the object (C) determines which function is called, leading to "C::fun()" being printed.

42. Answer: A) In Base
Explanation: The code explicitly uses the scope resolution operator (Base::show()) to call the show() function of the base class.
Even though show() is a virtual function, specifying Base::show() bypasses the dynamic dispatch mechanism.
Therefore, the base class version of the function is called, resulting in "In Base" being printed.
This feature allows access to overridden base class functions when needed.

43. Answer: A) int *arr = new int [10];
Explanation: Dynamic array of pointers to integers requires an array where each element is a pointer to an integer.
int **arr = new int *[10]; correctly allocates memory for an array of 10 integer pointers.
Option B allocates memory for 10 integers, not pointers.
Option C has incorrect syntax because the type int* is not compatible with an array of pointers.
The correct form uses int **arr because it's an array of int*.

44. Answer: C) No Effect
Explanation: Deleting a NULL pointer is safe in C++.
The delete operator checks if the pointer is NULL before attempting to deallocate memory.
If the pointer is NULL, the operation simply does nothing, preventing crashes.
This is a key reason why setting pointers to NULL after deletion is considered good practice—it avoids dangling pointers and ensures safe repeated deletions.

45. Answer: D) More than one of the above (1, 2, and 3)
Explanation: Exception handling in C++ improves code readability and robustness.
It separates error-handling logic from the main code, making the code cleaner (Point 1).
Functions can catch specific exceptions while allowing others to propagate (Point 2).
C++ allows exceptions to propagate up the call stack, enabling centralized error management (Point 3).
Additionally, it helps in resource management with RAII (Resource Acquisition Is Initialization).

46. Answer: C) More than one of the above(Both 1 and 2)
Explanation: The try block is used to wrap code that may throw exceptions.
Point 1 is correct because any statement that might cause an exception should be placed inside the try block to catch potential errors.
Point 2 is also correct because when an exception occurs, the remaining code inside the try block is skipped, and control is transferred to the appropriate catch block.
Thus, any code that should not execute after an exception should also be inside the try block to ensure it's skipped during error handling.
This improves the program's robustness by controlling the flow of execution in case of errors.

47. Answer: A) Virtual functions allow dynamic binding at runtime.
Explanation: Virtual functions enable runtime polymorphism in C++.
They allow the program to decide which function to invoke at runtime (dynamic binding), based on the actual object type rather than the pointer/reference type.
Option B is incorrect because virtual functions do not have to be redefined in derived classes. If not overridden, the base class version is used.
Option C is incorrect because virtual functions are often accessed via base class pointers or references, which is the core idea of polymorphism.

48. Answer: A) Compiler Error when creating an object of the derived class
Explanation: A pure virtual function (declared as virtual void fun() = 0;) makes a class abstract.
If a derived class does not override all pure virtual functions, it remains abstract and cannot be instantiated.
Attempting to create an object of such a class will lead to a compiler error.
However, pointers or references to abstract classes can still be created, and objects of fully implemented derived classes can be assigned to them.

49. Answer: C) Base Constructor Derived Constructor Derived Destructor Base Destructor
Explanation: Step 1: new Derived() calls the Base constructor first, then the Derived constructor (due to constructor chaining).
Step 2: delete ptr; invokes the Derived destructor first, then the Base destructor because the destructor is declared as virtual in the base class.
Virtual destructors ensure proper cleanup of derived class resources when deleting via a base class pointer.

50. Answer: B) 5
Explanation: In this code, the new operator dynamically allocates memory for an integer and initializes it with the value 5. The pointer ptr holds the address of this allocated memory. When we use *ptr, it dereferences the pointer and returns the value stored at that address, which is 5. This demonstrates that the new operator can be used to both allocate memory and initialize primitive data types.

51. Answer: C) Compiler Error
Explanation: The code results in a compiler error because you cannot skip a middle argument when using default parameters. In the function call fun(12, , 2);, the second argument is missing. In C++, if you skip an argument, you must also skip all subsequent arguments. Valid calls would be fun(12) or fun(12, 2) but not partially skipping like in this case.

52. Answer: B) 14 13
Explanation: sizeof("abcdefghijklm ") returns 14 because it includes the null-terminator '\0' at the end of the string.
strlen("abcdefghijklm ") returns 13 as it counts the number of characters in the string excluding the null character.

53. Answer: A) 1 2 1 1
Explanation: ++x increments x to 2 (which is true). Since it's the left-hand side of the logical OR (||), and it's already true, the rest of the expression (++y && ++z) is not evaluated.
- Therefore, y and z remain unchanged at 1.
- The first cout prints 1 because the OR operation is true.
- The second cout prints 2 1 1 for x, y, and z, respectively.

54. Answer: A) Value of global x is 0; Value of local x is 10
Explanation: ::x refers to the global variable x, which is not explicitly initialized. In C++, global variables are automatically initialized to 0 if not explicitly assigned a value.
x in main() is a local variable initialized with 10.

55. Answer: C) Infinite Loop
Explanation: A for loop in C++ can run indefinitely if the condition is left blank, as it's treated as always true.
The above code will continuously print blank without stopping because there are no conditions to terminate the loop.
This is a classic example of an infinite loop.

56. Answer: C. hello!3
Explanation: The presence of the semicolon (;) right after the for loop makes it an empty loop body.
The loop runs three times with i taking values 0, 1, and 2. After the third iteration, i becomes 3, which breaks the loop.
Then, the cout statement executes, printing hello!3.
The key point is understanding that the semicolon ends the loop's operation, making the cout statement independent of the loop body.

57. Answer: C. 4
Explanation: The comma (,) operator evaluates expressions from left to right and returns the value of the last evaluated expression.
In (1, 4, 5, 6, 3), 3 is the final value after all expressions are evaluated.
Therefore, i = 1 + 3, resulting in i = 4.
The output will be 4. This concept often confuses beginners as they expect all values to contribute to the result, but only the last one matters in comma-separated expressions.

58. Answer: C. 50$60
Explanation: This program demonstrates the concept of embedded assignment.
First, a = 50 assigns the value 50 to a.
Then, b = (a = 50) + 10 evaluates to b = 50 + 10, making b = 60.
Hence, the output is 50$60.
Embedded assignments are often used to simplify code but should be used cautiously to maintain readability.

59. Answer: A. 120
Explanation: The program performs arithmetic operations on char data types.
In C++, char variables are treated as small integers (based on their ASCII values) when used in arithmetic operations.
Here, (a * b) / c becomes (30 * 40) / 10, which equals 120.
The result is stored in d, a char variable. When we use int(d) in cout, it prints the integer value 120 instead of its ASCII character.
This is a subtle point about typecasting in C++.

60. Answer: B. 4321
Explanation: The key concept here is the use of a static variable.
Static variables retain their values across multiple function calls.
The if (--i) condition decrements i and checks if it's non-zero. As long as i > 0, the program will recursively call main(10).
The sequence will be 4, 3, 2, 1, and when i becomes 0, the recursion stops.
The output is 4321, printed in decreasing order.
Without the static keyword, this program would result in infinite recursion.

61. Answer: C. Infinite Loop
Explanation: In this code, the variable i is not static, which means it will be re-initialized to 5 every time main(10) is called recursively.
The if (--i) statement decreases i to 4 and prints it.
The recursion continues infinitely because i is reset to 5 with each new call, preventing the termination condition (i == 0) from ever being met.
This causes an infinite loop and will eventually result in a stack overflow error due to uncontrolled recursion.
The key difference from the previous question is the absence of the static keyword.

62. Answer: C. Compiler Dependent
Explanation:This code results in undefined behavior due to the ambiguous use of x++ and ++x in the same expression.
Both x++ (post-increment) and ++x (pre-increment) modify x, and the sequence in which they are executed can vary across compilers.
Since C++ does not define a strict order for side effects in this scenario, different compilers may produce different outputs.
Some compilers might even give a warning or an error depending on strictness settings.
Undefined behavior means the program might crash, produce unexpected results, or even behave differently on the same machine with different compiler versions.

63. Answer: B. 0
Explanation: The continue statement causes the program to skip the remaining code inside the loop body and proceed directly to the next iteration.
Since continue is placed before cout, the cout statement is never executed.
The loop itself runs 10 times, but because the cout is never reached, the output is blank.
This is a good example of how control flow statements like continue can affect program behavior.

64. Answer: B. 11 followed by "label x"
Explanation: The program starts with n = 10.
In the first iteration of the for loop, n is incremented to 11, and cout << n prints 11.
Immediately after that, the goto x; statement transfers control to the label x, skipping the rest of the loop.
The do-while loop runs exactly once because of the break statement. It prints "label x".

65. Answer: B. Inside Switch case 1
Inside Switch case 2
Explanation: The switch statement matches choice = 1 with case 1.
Since there's no break after case 1, the execution falls through to case 2 and prints both statements.

66. Answer: E) None of the above
Explanation: A base class pointer can point to a derived class object. This is one of the key features of polymorphism in C++.
A derived class pointer cannot point to a base class object because the derived class may contain additional members not present in the base class, leading to undefined behavior.
A pointer to a derived class can be created just like any other pointer.
Hence, none of the statements are correct, making option E the right answer.

67. Answer: D) More than one of the above
Explanation: A) References in C++ must always refer to a valid object. They cannot be null, unlike pointers. B) Once a reference is bound to an object, it cannot be changed to refer to another object.
C) References are automatically dereferenced, which means you don't need to use the * operator like with pointers.
Since all options A, B, and C are correct, the answer is D (More than one of the above).

68. Answer: A) A constructor is called at the time of declaration of an object
Explanation:A constructor is a special member function that gets automatically called when an object of the class is declared.
B) is incorrect because the constructor is not called when the object is merely used; it's invoked at the moment of creation.
C) is incorrect because constructors are not called during class declaration—they are tied to object instantiation.
Hence, the correct answer is A.

69. Answer: C) constructor
Explanation:The virtual keyword in C++ is used to achieve runtime polymorphism and is applicable to member functions.

You cannot declare a constructor as virtual because constructors are responsible for initializing objects, and virtual behavior relies on an object's vtable, which isn't set up until after the constructor completes.
Virtual destructors, however, are allowed and are commonly used in polymorphic base classes.
Thus, the correct answer is C.

70. Answer: B) Construct a new object
Explanation: A constructor is a special function in C++ that is automatically invoked when an object is created. Its primary role is to initialize the object's data members.
A) is incorrect because classes are not "constructed"; they are defined.
C) is incorrect as functions are not constructed—they are defined and implemented.
Hence, the correct answer is B.

71. Answer: B) A problem in the calling function
Explanation: Exceptions in C++ are caused by runtime errors, such as division by zero, accessing invalid memory, or file I/O issues.
A) Missing semicolons are syntax errors, caught during compilation, not runtime.
C) Syntax errors also occur at compile-time, not as exceptions during program execution.
B) A problem in the calling function (like invalid arguments causing a runtime error) can lead to an exception.
Hence, the correct answer is B.

72. Answer: B) Polymorphism
Explanation: Polymorphism allows functions to behave differently based on the object type they're acting upon, even when accessed via a reference or pointer to a base class.
The compiler relies on the type of reference or pointer (static type), but due to virtual functions, the actual object type (dynamic type) is used at runtime to resolve calls.
This mechanism is known as dynamic binding.
Thus, the correct answer is B.

73. Answer: *C) int ShowData(void) const { / statements / }*
Explanation: A constant member function in C++ is declared using the const keyword after the function signature, like: int ShowData() const;
This ensures the function cannot modify any class data members (unless they are marked as mutable).
A and B make the return type constant, which is unrelated to constant member functions.
Thus, C is the correct answer.

74. Answer: A) Virtual function
Explanation: Late binding (or dynamic binding) means that the function to be called is determined at runtime. In C++, this is implemented using virtual functions. Operator functions and const functions are resolved at compile-time (early binding), so they are not relevant here.
Thus, the correct answer is A.

75. Answer: D) More than one of the above
Explanation:A) C++ performs static type checking at compile-time to ensure type safety.
B) C++ also supports dynamic type checking using features like dynamic_cast for polymorphic types.
C) Static member functions cannot be declared const because const applies to object instances, and static functions are not tied to any instance.
Hence, both A and B are correct, making D the right answer.

76. Answer: D) More than one of the above
Explanation: Reusability allows code to be used in multiple applications without rewriting, leading to:
Reduced testing time (A) because already tested modules don't need re-testing.
Lower maintenance costs (B) as reusable code reduces the need for extensive modifications.

C is incorrect because reusability doesn't directly impact compilation time.
Hence, both A and B are correct, making D the right answer.

77. Answer: A) this->x
Explanation: The this pointer is an implicit pointer available inside class member functions that points to the calling object.
To access a data member, you use the arrow operator (->), like:
this->x;
B and C are invalid because this is a pointer, not an object, so the dot operator (.) cannot be used.
Thus, the correct answer is A.

78. Answer: D) More than one of the above
Explanation: Static polymorphism (or compile-time polymorphism) occurs when function calls are resolved during compilation.
Operator overloading (A) allows redefining the behavior of operators for user-defined types.
Function overloading (B) enables multiple functions with the same name but different parameters.
Templates (C) support generic programming, allowing functions/classes to work with different data types.
Since all three (A, B, and C) are examples of static polymorphism, the correct answer is D.

79. Answer: B) Both I and II are false
Explanation: Statement 1 is false because not all operators can be overloaded. For example, ::, .*, ., ?:, and sizeof cannot be overloaded.
Statement 2 is false because operator overloading doesn't change the fundamental meaning of operators. It only extends their functionality for user-defined types.
Thus, the correct answer is B.

80. Answer: D) Base class object will call the base class function, and derived class object will call the derived class function
Explanation: If both the base and derived classes have functions with the same signature (without using virtual), this is known as function hiding.
The function called depends on the type of the object:
Base class object calls the base class function.
Derived class object calls the derived class function.
If the base class function were virtual, the behavior would depend on runtime polymorphism.
Thus, the correct answer is D.

81. Answer: C) Protected data members
Explanation: Public members (A) are accessible from anywhere (outside the class, inside derived classes, etc.).
Private members (B) are accessible only within the class where they are declared, not even in derived classes.
Protected members (C) are accessible only within the class and its derived classes, which makes them specific to the class hierarchy chain.
Thus, the correct answer is C.

82. Answer: D) More than one of the above
Explanation: Virtual calls are generally resolved at runtime using dynamic dispatch.
However, inside constructors and destructors (A and B), virtual calls are resolved at compile-time because the object is not fully constructed (or already partially destructed), so polymorphism doesn't apply.
Calls made from main() are not inherently tied to virtual dispatch unless a polymorphic call is explicitly made.
Thus, both A and B are correct, making D the right answer.

83. Answer: D) More than one of the above
Explanation: Inline functions are expanded in place where they are called, avoiding function call overhead, which can speed up execution (A) for small functions.

However, inlining increases the code size (C) because the function code is duplicated at each call site, which can lead to code bloat.
B is incorrect because it doesn't inherently slow down execution.
Thus, both A and C are correct, making D the right answer.

84. Answer: C) virtual void Display(void) = 0;
Explanation: A pure virtual function is declared using the syntax:
virtual void Display() = 0;
This indicates that the function must be overridden in any derived class, making the class abstract.
A is invalid because {0} is not the correct syntax.
B is incorrect as the function signature is missing.
Thus, the correct answer is C.

85. Answer: E) None of the above
Explanation: The correct header file that defines cin and cout in C++ is iostream, not iostream.h.
istream.h and ostream.h are non-standard headers used in older C++ versions.
iomanip.h is used for input/output formatting, not for declaring cin/cout.
Thus, the correct answer is E.

86. Answer: B) operator
Explanation: In C++, the operator keyword is used to overload operators. friend is used for friend functions but not for operator overloading directly.
overload is not a keyword in C++.

87. Answer: D) More than one of the above
Explanation: A class without a name is known as an anonymous class.
Anonymous classes cannot have constructors or destructors because there's no way to reference them directly for instantiation or destruction.
However, they are allowed in C++ (especially as local types or within other classes).
Thus, both A and B are correct, making D the right answer.

88. Answer: B) Multiple inheritance
Explanation: Multiple inheritance occurs when a class inherits from more than one base class, as shown in the code where A inherits from both X and Y.
Multilevel inheritance involves a chain of inheritance (A → B → C).
Hybrid inheritance is a combination of multiple and multilevel inheritance.
Thus, the correct answer is B.

89. Answer: C) Only I is correct
Explanation: In Procedure-Oriented Programming (POP), function calls are usually resolved at compile-time because it lacks features like polymorphism.
In Object-Oriented Programming (OOPs), function calls can be resolved at both compile-time (e.g., function overloading) and runtime (e.g., virtual functions).
Thus, only statement I is true, making C the correct answer.

90. Answer: E) None of the above
Explanation: In C++, the valid visibility (access) specifiers for inheritance are:
public inheritance (retains original access specifiers).
protected inheritance (public members become protected).
private inheritance (public and protected members become private).
All the options listed are valid access specifiers in inheritance.
Thus, the correct answer is E.

91. Answer: B) Structures can have functions as members
Explanation: In C++, structures (struct) can have member functions, unlike in C.
A is incorrect because classes can have protected data members.
C is incorrect because class members are private by default (unlike structs, which are public by default).

92. Answer: D) Making at least one member function a pure virtual function
Explanation: An abstract class in C++ is created by defining at least one pure virtual function, like this:

```
class AbstractClass {
   virtual void display() = 0;  // Pure virtual function
};
```

Pure virtual functions enforce that derived classes must override them.
The virtual keyword alone does not make a class abstract.
Thus, the correct answer is D.

93. Answer: C) private
Explanation: In C++, the default access specifier for class members is private.
This means that if no access specifier is provided, class members are treated as private.
For structures (struct), the default access specifier is public.
Thus, the correct answer is C.

94. Answer: D) More than one of the above
Explanation: A is correct: A static member function can only access static data members because it doesn't operate on specific instances.
B is correct: A static data member is shared across all objects of the class, meaning it has a single copy for all instances.
C is incorrect unless accessed through proper scope resolution, e.g., ClassName::staticVar.
Thus, both A and B are correct, making D the right answer.

95. Answer: B) When a class inherits from two classes with the same method
Explanation: The "diamond problem" occurs in programming when a class inherits from two classes that both inherit from a common base class, and these parent classes have the same method. This creates ambiguity about which method the subclass should inherit.
The term "diamond problem" comes from the diamond-shaped hierarchy:

```
  A
 / \
B   C
 \ /
  D
```

Class B and Class C both inherit from A, and D inherits from both B and C.
If A has a method (e.g., display()), and both B and C override it, which version will D inherit? This is the ambiguity.

96. Answer: A) Compilation error
Explanation: In object-oriented programming (OOP) languages like Java and C++, methods are typically called on instances (objects) of a class.
If a programmer doesn't create an object and tries to call a non-static method directly using the class name, it will result in a compilation error because non-static methods require an object context.
To call a method without creating an object, the method should be declared as static.
Thus, the correct answer is A.

97. Answer: C) The data stored within the object
Explanation: In OOP, the state of an object refers to the data or values stored in its attributes (variables).

For example, consider a Car object:

```
class Car {
   String color;
   int speed;
}
```

Here, color and speed represent the state of the Car object because they store data that defines the current condition of the car.

The behavior of an object refers to the methods/functions it can perform, such as drive() or brake().

98. Answer: B) A runtime entity

Explanation: An object in programming is a runtime entity that represents an instance of a class.

Objects have three key characteristics:

1. State: The data it holds.
2. Behavior: The functions or methods it can perform.
3. Identity: A unique address in memory

99. Answer: A) A class is a blueprint, while an object is an instance of a class

Explanation: A class in OOP is like a blueprint or template that defines the structure and behavior (methods and properties) of objects.

An object is an instance of a class, created using the class definition at runtime.

100. Answer: E) None of the above

Explanation: The provided code defines a class named Counter, but it does not contain a main() method to execute the code.

There are no syntax errors, so no compilation error will occur.

Since no object is created and no method is invoked, no runtime error will occur either.

The program doesn't output 0 because no cout is used to display any value.

Therefore, since nothing happens (no errors, no output), the correct answer is E) None of the above.